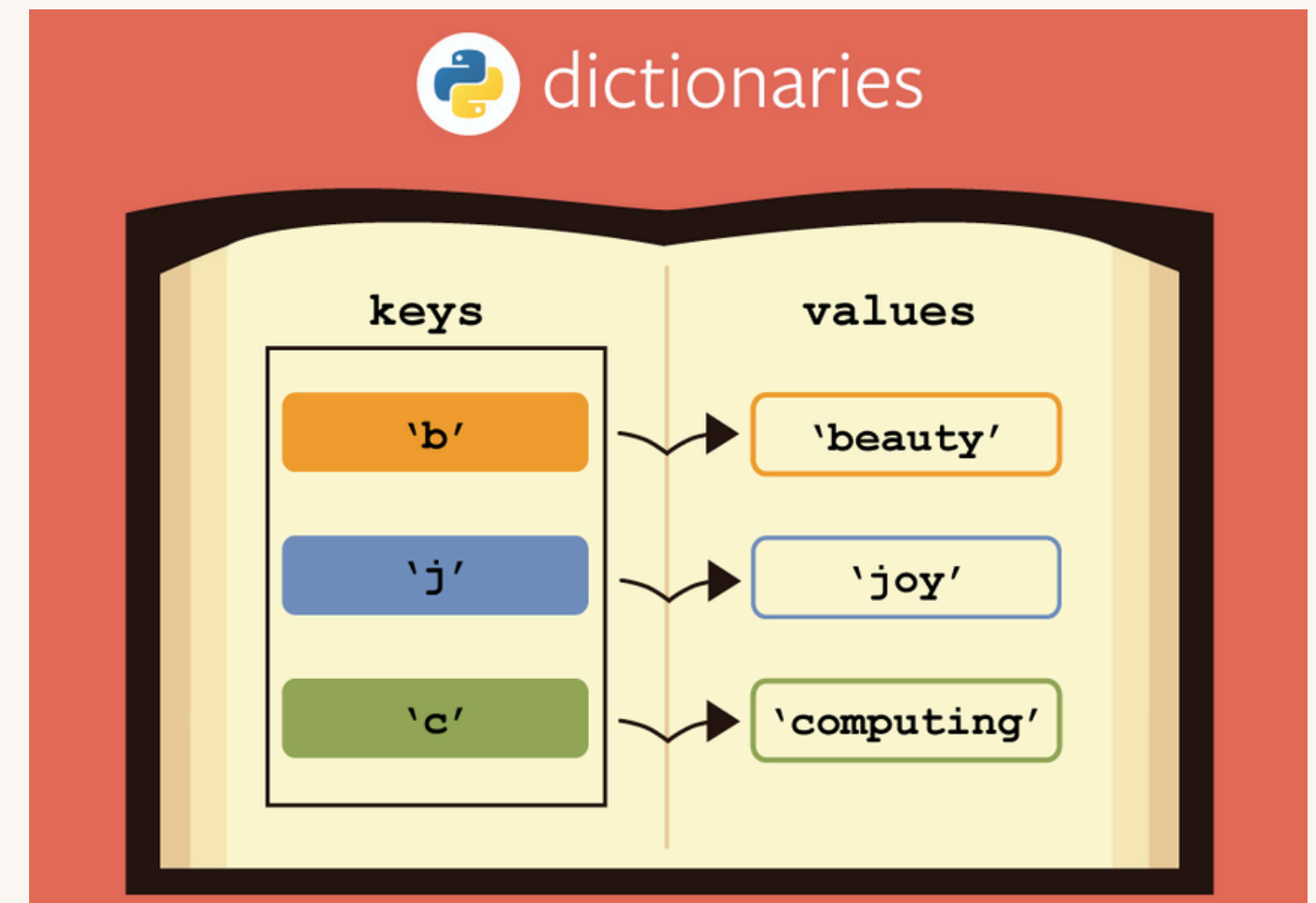


Словари





Основные термины, словарь, ключ, значение

Словарь в Python — упорядоченная коллекция произвольных объектов с доступом по ключу. Иногда его еще называют ассоциативным массивом или хеш-таблицей. Как известно, объектом в Python является абсолютно всё, а каждый объект относится к какому-либо типу данных. Типы данных бывают изменяемые и неизменяемые (англ. mutable и immutable). Словари — изменяемый тип.

Ключ — имя для доступа к ячейке с данными. Похож на индекс в списках и строках. Ключом в словаре могут быть любые неизменяемые типы данных, например: строки, числа, булев тип и т. д. Два элемента не могут иметь одинаковый ключ.

Значение — всё, что мы положили в ячейку с определенным ключом.

Значением ключа в словаре может быть любой тип данных: строки, числа, булево значение, списки.



Словарь

Словарь чем то похож на список, ведь разделяется запятыми но в каждом элементе есть пара ключ: значение, после элемента ставится запятая, и идет следующий элемент, по краям словаря ставятся фигурные скобки.

Python словарь

Diagram illustrating the structure of a Python dictionary:

```
pi_dict = { 1 : 'Яблоко' , 2 : 'ПлюсОдин' }
```

The diagram labels the components of the dictionary:

- Ключ** (Key) points to the numbers `1` and `2`.
- Значение** (Value) points to the strings `'Яблоко'` and `'ПлюсОдин'`.
- Элемент 1** (Element 1) points to the first pair `1 : 'Яблоко'`.
- Элемент 2** (Element 2) points to the second pair `2 : 'ПлюсОдин'`.



Получение значений из словаря

Для получения значений из словаря используется похожая на списки формула, но в квадратных скобках мы указываем ключ. У списков мы указывали индекс, и если при создании списков индексы присваиваются самим Python, то в словарях у нас есть контроль над ключами

Неупорядоченные коллекции уникальных значений хранятся в парах (Ключ-Значение)

$d = \{ 'a' : 10 , 'b' : 20 , 'c' : 30 \}$

↑ ↑ ↑
d['a'] d['b'] d['c']

- **Неупорядоченные** элементы в словаре хранятся без значения индекса
- **Уникальные** ключи в словарях должны быть уникальными
- **Переменные** разрешено добавлять/изменять/удалять Ключ: Значение после создания



Создание словаря

Пустой словарь можно создать при помощи фигурных скобок {} присвоив их к некой переменной:

```
my_dict = {}
```

Также мы **можем создать словарь уже с некоторыми значениям**, для этого необходимо в фигурных скобках создать пары ключ: значение (key: value):

```
my_dict = {"Петр": 74957556983, "Мария": 74957556982}
```

```
my_dict = {"Петр": "Имя человека", "Python": "Язык программирования"}
```

Создать при помощи функции dict() от слова dictionary которое переводится как словарь:

```
my_dict = dict(Петр = 74957556983, Мария = 74957556982)
```

Обратите внимание что имя ключа задано в данном методе без кавычек!

Данные варианты между собой не сильно различаются ни по популярности ни по скорости, так что можно использовать оба



Получение значений по ключу

Для получения информации из словаря мы должны обратиться к ключу, для этого мы пишем имя словаря, ставим квадратные скобки и в квадратных скобках пишем ключ, и нам вернется значение этого ключа

```
my_dict = {"Петр": 74957556983, "Мария": 74957556982}
print(my_dict["Петр"])
print(my_dict["Мария"])

my_dict = {"Петр": "Имя человека", "Python": "Язык программирования"}
print(my_dict["Python"])
```

>>> 74957556983
>>> 74957556982

>>> Язык программирования

Что может быть ключом - ключом в словаре могут быть любые не изменяемые типы данных, например: строки, числа, булевый тип и т.д.

```
my_dict = {"Петр": "Имя человека", 59: "Это число", 3.14: "Число Pi", True: 1}
print(my_dict)
```

v v v

```
{'Петр': 'Имя человека', 59: 'Это число', 3.14: 'Число Pi', True: 1}
```



Получение значений по ключу

В качестве специальных значений в словарях могут храниться некоторые дополнительные пояснения например:

```
explanations = {True: "Все верно", False: "Не верно"}

if 3 > 2:
    print(explanations[True])
else:
    print(explanations[False])
```

>>> Все верно

Такую запись можно сконвертировать на вывод сразу по какому-то выражению

```
explanations = {True: "Все верно", False: "Не верно"}

print(explanations[3 > 2])
print(explanations[3 < 2])
```

>>> Все верно
Не верно

Что может быть значением - тут нет никаких ограничений, значением может быть любой тип данных: строки, числа, булевый тип, списки, словари, множества, кортежи (**самое главное чтобы это был поддерживаемый Python тип данных**)



Работа со словарями

Словарь это и изменяемый тип данных!

Это значит что после его создания мы можем изменять в нем значения, а также добавлять или удалять пары ключ: значения внутри них, например:

- добавление пары ключ: значение

```
my_dict = {"cat": "Котейка"}  
my_dict["dog"] = "Собака"  
print(my_dict)
```

>>> {'cat': 'Котейка', 'dog': 'Собака'}

- изменение значения по ключу

```
my_dict["cat"] = "Кошка"  
print(my_dict)
```

>>> {'cat': 'Кошка', 'dog': 'Собака'}

- удаление пары ключ: значение

```
del my_dict["cat"]  
print(my_dict)
```

>>> {'dog': 'Собака'}



Работа со словарями

Как еще можно добавить пары ключ: значение в словарь? Для этого существует метод **.update()**, по своему использованию он похож на методы множеств: перед методом мы указываем тот словарь который мы хотим расширить, а в скобки передается словарь с новыми парами ключ: значение.

```
my_animals_dict = {'cat': 'Кошка', 'dog': 'Собака'}
shop_animals_dict = {'parrot': 'Попугай', 'turtle': 'Черепаха'}
my_animals_dict.update(shop_animals_dict)
print(my_animals_dict)
print(shop_animals_dict)
```

v v v

```
{'cat': 'Кошка', 'dog': 'Собака', 'parrot': 'Попугай', 'turtle': 'Черепаха'}
{'parrot': 'Попугай', 'turtle': 'Черепаха'}
```

Действительно, мы обновили словарь `my_animals_dict`, добавив туда значения из переданного в скобки словаря `shop_animals_dict`, обратите внимание что переданный в скобки словарь не изменился

Но надо быть осторожным, пользуясь этим методом, поскольку, как видно, во время его использования переменная которая содержит базовый словарь перезаписывается.



Работа со словарями

Что делать если мы хотим изменить именно ключ? Поскольку напрямую ключи в словарях изменить не получится это можно сделать путем удаления старой пары, ключ: значение и замены ее на новую пару, как это можно сделать?

Есть 2 метода:

- путем присвоения новому ключу, значения старого ключа, после чего удаляем старый ключ

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
my_dict['die Katze'] = my_dict['cat']  
del my_dict['cat']  
my_dict['der Hund'] = my_dict['dog']  
del my_dict['dog']  
print(my_dict)
```

>>> {'die Katze': 'Кошка', 'der Hund': 'Собака'}

- 2й при помощи метода .pop(), он не только удаляет что-либо но может сообщить нам что удаляет

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
my_dict['die Katze'] = my_dict.pop('cat')  
my_dict['der Hund'] = my_dict.pop('dog')  
print(my_dict)
```

>>> {'die Katze': 'Кошка', 'der Hund': 'Собака'}



Как избежать ошибок

Если мы обратимся к словарю по несуществующему ключу мы получим ошибку вида:

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
print(my_dict['bird'])  
>>> print(my_dict['bird'])  
~~~~~^~~~~~  
KeyError: 'bird'
```

Которая нам напрямую и сообщает что в данном словаре у нас ошибка в ключевом слове (либо мы его неправильно написали: регистр, пробелы, грамматика) либо его и вовсе нет.

Обойти данную ошибку очень просто с помощью уже известной вам конструкции if - else, и оператора in.

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
word = 'bird'  
if word in my_dict:  
    print(my_dict[word])  
else:  
    print("Не знаю таких слов")  
>>> Не знаю таких слов
```

Поэтому перед работой с незнакомым нам словарем, или если мы что-то забыли, или хотим проверить наличие какой-то пары ключ: значение, или мы по одному и тому же запросу можем получить разные словари, необходимо проводить подобную проверку.



Как избежать ошибок

Также мы можем использовать методы `.get()` и `.setdefault()` разберемся на примере:

- метод `.get()` возвращает значение искомого ключа, если ключ не найден то метод вернет `None`:

```
my_animals_dict = {'cat': 'Кошка', 'dog': 'Собака'}
get_animal = my_animals_dict.get('cat')
get_animal_1 = my_animals_dict.get('parrot')
print(my_animals_dict)
print(f"Значение переданного ключа {get_animal}")
print(f"Значение переданного ключа {get_animal_1}")
```

```
>>> {'cat': 'Кошка', 'dog': 'Собака'}
Значение переданного ключа Кошка
Значение переданного ключа None
```

При использовании метода `.get()`, сам словарь не изменяется, если ключ найден, то мы получаем значение, если нет то `None`

- метод `.setdefault()` возвращает значение искомого ключа, если ключ не найден то метод создает ключ который мы передали ему в скобках, созданному ключу присваивается значение `None`:

```
my_animals_dict = {'cat': 'Кошка', 'dog': 'Собака'}
get_animal = my_animals_dict.setdefault('cat')
get_animal_1 = my_animals_dict.setdefault('parrot')
print(my_animals_dict)
print(f"Значение переданного ключа {get_animal}")
print(f"Значение переданного ключа {get_animal_1}")
```

```
>>> {'cat': 'Кошка', 'dog': 'Собака', 'parrot': None}
Значение переданного ключа Кошка
Значение переданного ключа None
```

При использовании метода `.setdefault()`, в словарь был добавлен ключ со значением `None`. Если ключ найден, то мы получаем его значение, а вот если ключа в словаре не было то он создается и ему присваивается значение `None`



Использование в расчетах

Например мы хотим узнать стоимость фруктов в магазине, для этого можно написать следующую программу для расчета.

```
fruits = {  
    "яблоки": 100,  
    "груши": 200,  
    "мандарины": 300,  
}  
  
fruit = input("Выберете фрукт: ")  
weight = float(input("вес в граммах: "))  
  
price = fruits[fruit] * weight / 1000  
print(f"Стоимость: {price} руб.")
```

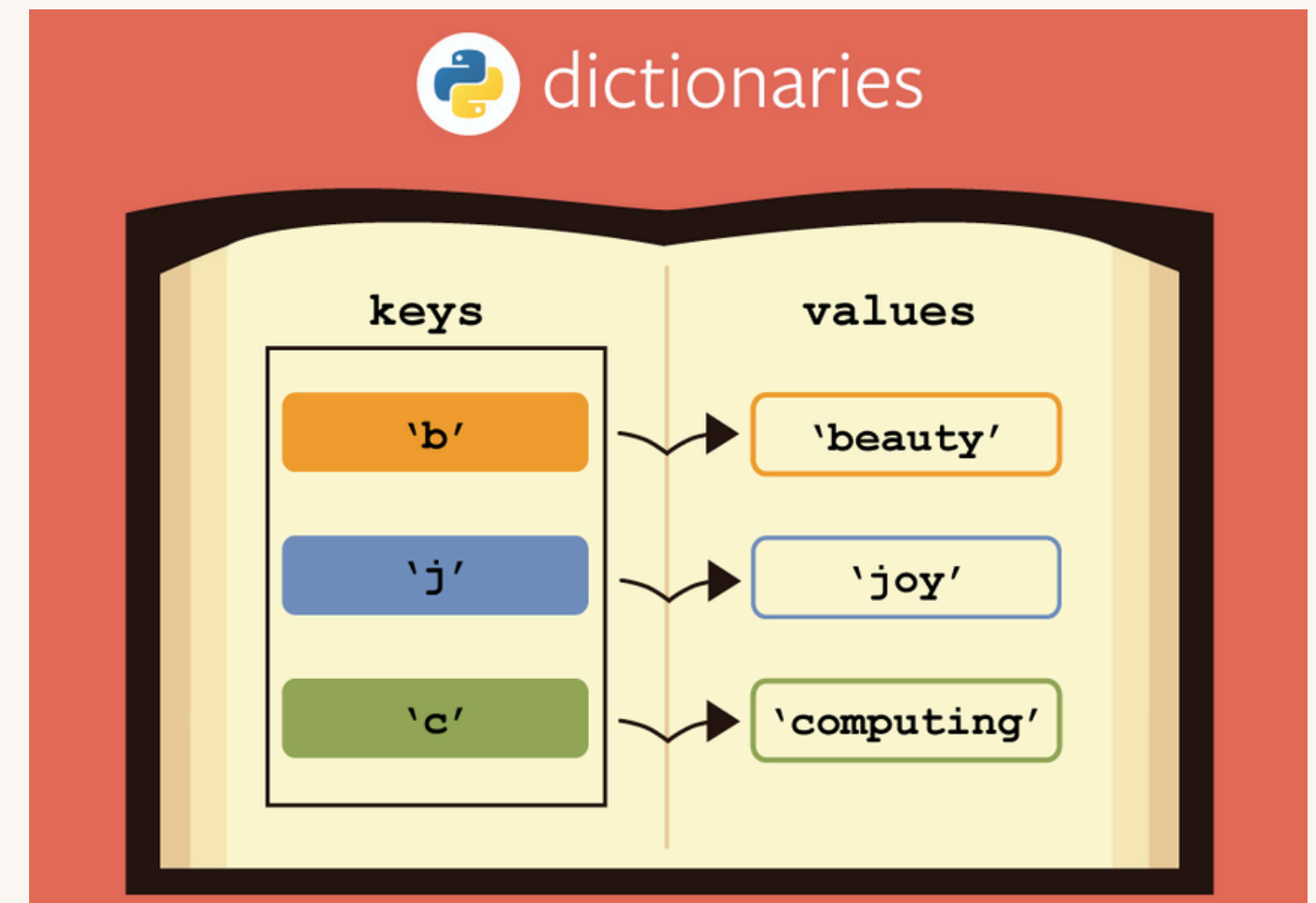
>>>

Выберете фрукт: мандарины
вес в граммах: 1500
Стоимость: 450.0 руб.

Выберете фрукт: яблоки
вес в граммах: 550
Стоимость: 55.0 руб.

Как мы видим в зависимости от выбранного фрукта, и введенного веса, мы можем узнать конечную стоимость определенного продукта.

Keys, Values, Items





Методы словарей

Как было сказано ранее **внутри словаря есть неизменяемые ключи по которым мы можем получать значения соответствующие этим ключам**, но иногда может возникнуть необходимость в получении всех ключей или всех значений в виде списка.

Например мы хотим получить только список наших продуктов или только перечень цен (казалось бы список продуктов понятно зачем а зачем цены без указания за что, мне интересно ваше мнение)

Для получения необходимых данных в этом случае у словарей есть специальные методы:

- **.keys()** который возвращает список ключей
- **.values()** который возвращает список значений

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
my_dict_keys = my_dict.keys()  
my_dict_values = my_dict.values()  
print(my_dict_keys)  
print(my_dict_values)  
  
>>> dict_keys(['cat', 'dog'])  
>>> dict_values(['Кошка', 'Собака'])
```

Обратите внимание python сам нам сообщил то, что он выдал, я ничего дополнительно не передавал.



Методы словарей

Также можно перебрать словарь при помощи уже известного нам цикла `for ... in ...`

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
for k in my_dict:  
    print("Это ключ", k)  
    print("Это значение", my_dict[k])
```

>>> Это ключ cat
Это значение Кошка
Это ключ dog
Это значение Собака

Если же нам нужно получить список ключей и значений можно поступить так:

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
keys_list = [k for k in my_dict]  
values_list = [my_dict[k] for k in my_dict]  
print(keys_list)  
print(values_list)
```

>>> ['cat', 'dog']
['Кошка', 'Собака']

Также можно воспользоваться обычным циклом, только необходимо сначала создать пустые списки которые вы заполните при помощи метода `.append()`



Методы `.keys()` `.values()` в цикле

Ранее упомянутые методы `.keys()` и `.values()` можно использовать и в цикле т.к. по своей сути они возвращают нам списки ключей или значений словаря

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}
```

```
for key in my_dict.keys():  
    print(f"Это ключ: {key}")
```

>>>

Это ключ: cat

Это ключ: dog

Это ключ: Кошка

Это ключ: Собака

```
for value in my_dict.values():  
    print(f"Это ключ: {value}")
```



Метод `.items()` (более полезный метод для словаря)

Метод `.items()` используется для того чтобы при переборе цикла **сразу получить нужную нам пару и ключа и значения**.

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
for key, value in my_dict.items():  
    print(f"Ключ: {key}, Значение: {value}")
```

>>> Ключ: cat, Значение: Кошка
>>> Ключ: dog, Значение: Собака

Так как метод `.items()` возвращает **2 значения**, то и **перебираем мы 2 переменные: `key` и `value`**

Данный метод прекрасно подходит и для создания списка ключей и списка значений

```
my_dict = {'cat': 'Кошка', 'dog': 'Собака'}  
keys_list = []  
values_list = []  
  
for key, value in my_dict.items():  
    keys_list.append(key)  
    values_list.append(value)  
  
print(keys_list)  
print(values_list)
```

>>> ['cat', 'dog']
>>> ['Кошка', 'Собака']



Методы словарей применение на практике

Например компания пошла в ресторан и они решили поделить счет. Выведем список гостей, итоговую сумму за счет и сколько нужно заплатить каждому (вот решили счет поделить поровну).

```
guests = {  
    "Сергей": 1600,  
    "Андрей": 2200,  
    "Екатерина": 2000,  
    "Александр": 2300,  
    "Диана": 2400,  
}  
total = 0  
guest_names = ", ".join(guests.keys())  
  
for check in guests.values():  
    total += check  
  
print(f"В ресторане были {guest_names}")  
print(f"Всего наели напили {total}")  
print(f"Оплатить каждому {total / len(guests)}")
```

>>>

```
В ресторане были Сергей, Андрей, Екатерина, Александр, Диана  
Всего наели напили 10500  
Оплатить каждому 2100.0
```